

TP — Unit Testing

Continuación sobre Patrones de Diseño

Materia	Ingeniería de Software
Tema	Unit Testing: Fakes, Stubs y Mocks
Base	TP anterior — Patrones de Diseño (Singleton · Strategy · Observer)
Modalidad	1 trabajo por equipo (mismo equipo que el TP anterior)
Tiempo	1 semana desde la presentación del práctico
Lenguaje	El mismo que usaron en el TP anterior

Descripción general

Este trabajo parte del sistema de transporte que implementaron en el TP anterior. No se espera un sistema nuevo: el foco está en aprender a escribir tests unitarios sobre código ya existente y sobre una pequeña refactorización que se pide en la Parte 1.

La estructura del código debe seguir reflejando los patrones del TP anterior. Esta práctica agrega tests y una refactorización puntual, no reemplaza el diseño previo.

Parte 1 — Refactorización: AlertService

Actualmente `AlertObserver` tiene dos responsabilidades mezcladas: **decidir si hay que alertar** (lógica de umbrales) y **actuar cuando hay que alertar** (loggear). Esto dificulta testear cada parte por separado.

Se pide extraer la lógica de decisión a una nueva clase `AlertService` con la siguiente interfaz mínima:

```
// Java
public interface AlertService {
    boolean shouldAlertCost(double cost);
    boolean shouldAlertETA(int eta);
}

// Implementación concreta
public class ThresholdAlertService implements AlertService {
    public ThresholdAlertService(double maxCost, int maxEta) { ... }
    ...
}
```

Resultado esperado

- `AlertObserver` recibe un `AlertService` por constructor y lo delega para decidir si debe loggear.
- `AlertObserver` ya no conoce los umbrales directamente.
- `ThresholdAlertService` encapsula toda la lógica de comparación.

Esta separación ilustra el principio **Single Responsibility**: cada clase tiene una única razón para cambiar.

Parte 2 — Tests unitarios

Se piden tres grupos de tests. Cada grupo tiene un objetivo distinto y usa una técnica diferente.

2.1 — Tests de AlertService (sin dependencias)

Testear `ThresholdAlertService` de forma aislada, sin necesidad de mocks ni fakes. Es una clase pura: dado un input, devuelve un resultado determinístico.

Tests requeridos:

1. Costo por debajo del umbral → `shouldAlertCost` retorna `false`.
2. Costo exactamente en el umbral → definir y documentar el comportamiento esperado.
3. Costo por encima del umbral → `shouldAlertCost` retorna `true`.
4. ETA por debajo del umbral → `shouldAlertETA` retorna `false`.
5. ETA por encima del umbral → `shouldAlertETA` retorna `true`.

Estos tests no requieren ningún doble de prueba. Son el ejemplo más simple de unit test: instanciar la clase, llamar un método, verificar el resultado.

2.2 — Tests de AlertObserver con Fake

Testear `AlertObserver` de forma aislada usando un **fake manual** de `AlertService`. Un fake es una implementación alternativa simple escrita a mano, que permite controlar el comportamiento en el test.

Ejemplo de fake:

```
public class AlwaysAlertService implements AlertService {  
    @Override  
    public boolean shouldAlertCost(double cost) { return true; }  
  
    @Override  
    public boolean shouldAlertETA(int eta) { return true; }  
}
```

Tests requeridos:

1. Usando `AlwaysAlertService`: verificar que `AlertObserver` logea cuando se notifica con cualquier snapshot.
2. Usando un fake que siempre retorna `false`: verificar que `AlertObserver` **no** logea nada.

Para verificar lo que se loggó, pueden hacer un fake del Logger también (un `FakeLogger` que guarda los mensajes en una lista en lugar de imprimirlos). Esto evita que los tests dependan de la salida de consola.

2.3 — Tests de AlertObserver con Mock

Repetir los mismos escenarios de la sección 2.2, pero usando la **biblioteca de mocks** disponible en su lenguaje (Mockito para Java, `unittest.mock` para Python, etc.) en lugar del fake manual.

Tests requeridos:

1. Verificar que cuando `shouldAlertCost` retorna `true`, `AlertObserver` llama al logger con `logWarning`.
2. Verificar que cuando `shouldAlertETA` retorna `true`, `AlertObserver` llama al logger con `logError`.
3. Verificar que cuando ambas condiciones son `false`, el logger **no es llamado** en ningún momento.

La diferencia clave respecto al fake: el mock permite **verificar interacciones** (si un método fue llamado, cuántas veces, con qué argumentos), no solo el resultado final.

Entregables

- `AlertService` e implementación `ThresholdAlertService`.
- `AlertObserver` refactorizado para recibir `AlertService` por constructor.
- Suite de tests con los tres grupos descriptos (2.1, 2.2, 2.3).
- Al menos un fake manual y al menos un test usando la biblioteca de mocks.
- Todos los tests deben pasar sin errores.
- README actualizado indicando cómo correr los tests.

Preguntas orientadoras

No son obligatorias de responder por escrito, sirven para ayudar con el razonamiento:

1. **Fake vs Mock** — Escribieron el mismo test de dos formas distintas. ¿Cuándo conviene usar un fake manual y cuándo conviene usar un mock de biblioteca? ¿Qué información extra da el mock que el fake no da?
2. **Diseño testeable** — ¿Por qué fue necesario extraer `AlertService` para poder testear `AlertObserver` de forma aislada? ¿Qué habría pasado si intentaban testearlo sin esa separación?
3. **Singleton y testing** — El Logger es un Singleton. ¿Qué problema trae eso al momento de testearlo? ¿Cómo lo resolvieron en los tests?